

## ۱. چرا به ایندکس گذاری نیاز داریم

در یک پایگاه داده رابطه‌ای، داده‌های هر جدول معمولاً به همان ترتیبی روی دیسک ذخیره می‌شوند که درج شده‌اند. وقتی موتور پایگاه داده بخواهد رکوردی را بر اساس یک شرط پیدا کند، در نبود هر ساختار کمکی، ناچار است تمام ردیف‌های جدول را یکی‌یکی بخواند و شرط را روی هر کدام بسنجد. برای جدول‌های کوچک این روش مشکلی ایجاد نمی‌کند، اما با رشد تعداد ردیف‌ها، به‌خصوص در جدول‌هایی مثل سفارش یا پرداخت که در یک فروشگاه آنلاین واقعی می‌توانند میلیون‌ها ردیف داشته باشند، هزینه این جست‌وجوی خطی به‌سرعت غیرقابل قبول می‌شود.

ایندکس ساختار داده‌ای جداگانه‌ای است که پایگاه داده علاوه بر جدول اصلی نگهداری می‌کند و به آن اجازه می‌دهد بدون خواندن کل جدول، مستقیماً به ردیف‌های موردنظر برسد. متداول‌ترین نوع ایندکس در PostgreSQL، B-Tree است؛ ساختاری درختی و متوازن که مقادیر یک یا چند ستون را به‌صورت مرتب نگه می‌دارد و امکان جست‌وجو، مقایسه بازه‌ای و مرتب‌سازی را در زمانی نزدیک به لگاریتمی فراهم می‌کند، به‌جای زمان خطی جست‌وجوی کامل جدول.

نکته‌ای که با انجام این گام پروژه به آن میرسیم این است که ایندکس گذاری رایگان نیست. هر ایندکس اضافه، حجم دیتابیس را افزایش می‌دهد و مهم‌تر از آن، هر عملیات INSERT، UPDATE یا DELETE روی جدول باید علاوه بر خود جدول، تمام ایندکس‌های مرتبط با آن را نیز به‌روزرسانی کند. بنابراین ایندکس گذاری بی‌رویه یا حدسی، به‌جای بهبود عملکرد، می‌تواند سرعت نوشتن داده را کاهش دهد و فضای ذخیره‌سازی را بدون دلیل مصرف کند. به همین علت، پیش از تعریف هر ایندکس، سه پرسش پایه باید پاسخ داده شود:

- این ستون در کدام کوئری‌های واقعی سیستم، در شرط WHERE، در ORDER BY یا در JOIN استفاده می‌شود؟
- آیا این ستون تنوع داده کافی دارد؟ ستونی که تنها دو یا سه مقدار ممکن دارد و هر کدام سهم بزرگی از جدول را تشکیل می‌دهند، از ایندکس سود چندانی نمی‌برد، چون بهینه‌ساز پایگاه داده در چنین حالتی معمولاً ترجیح می‌دهد کل جدول را بخواند.
- آیا این ستون از قبل، به‌واسطه یک کلید اصلی، کلید خارجی یا محدودیت یکتایی (Unique)، به‌طور ضمنی ایندکس شده است؟ این سوال مهم است زیرا در PostgreSQL، تعریف Primary Key یا Unique به‌طور خودکار یک ایندکس B-Tree می‌سازد؛ بنابراین تعریف مجدد ایندکس روی این ستون‌ها تکراری و بی‌فایده است.

در ادامه این گزارش، شش پرس‌وجوی تستی تعریف‌شده در فاز سوم پروژه یک‌به‌یک بررسی می‌شوند. برای هر کدام، مسئله عملکردی که در نبود ایندکس مناسب پیش می‌آید شرح داده شده و در ادامه، ایندکسی که برای رفع آن طراحی شده همراه با استدلال کامل ارائه می‌شود.

### مسئله ۱: بازیابی سفارش‌های یک مشتری مشخص و مرتب‌سازی بر اساس تاریخ

پرس‌وجوهای شماره ۱ و ۲ از فاز سوم، هر دو روی جدول Order اجرا می‌شوند و ابتدا با شرط `customer_id = مقدار مشخص`، سفارش‌های یک مشتری را جدا می‌کنند و سپس نتیجه را بر اساس `order_date` به صورت نزولی مرتب می‌کنند؛ پرس‌وجوی دوم علاوه بر این، با `LIMIT ۵` تنها پنج سفارش اخیر را برمی‌گرداند.

در ساختار فعلی جدول Order، ستون `customer_id` بخشی از یک کلید خارجی ترکیبی همراه با `customer_postal_code` است. این یعنی ایندکس ضمنی موجود روی این دو ستون با هم تعریف شده، نه با `order_date`. در نتیجه وقتی سیستم بخواهد سفارش‌های یک مشتری را بر اساس تاریخ مرتب کند، مجبور است تمام سفارش‌های آن مشتری را (که ممکن است پراکنده در جدول باشند) پیدا کند و سپس آن‌ها را در حافظه مرتب‌سازی کند؛ عملیاتی که با رشد تعداد سفارش‌ها کند و پرهزینه می‌شود.

### راه‌حل

برای این مسئله یک ایندکس ترکیبی روی (`customer_id, order_date DESC`) تعریف شده است:

```
CREATE INDEX IDX_Order_Customer_Date
ON "Order" (customer_id, order_date DESC);
```

در این ایندکس، `customer_id` به عنوان ستون اول باعث می‌شود پایگاه داده بتواند سریع به بخش مربوط به یک مشتری خاص برسد، و چون `order_date` با ترتیب نزولی به عنوان ستون دوم در همان ایندکس ذخیره شده، رکوردهای هر مشتری از قبل به ترتیب زمانی نزولی مرتب هستند. نتیجه این است که برای پرس‌وجوی دوم (پنج سفارش اخیر)، سیستم نیازی به مرتب‌سازی جداگانه ندارد و می‌تواند به طور مستقیم از ابتدای بخش مربوط به آن مشتری در ایندکس، پنج رکورد اول را بخواند و متوقف شود.

### مسئله ۲: گزارش گیری کلی بر اساس بازه زمانی، بدون فیلتر مشتری خاص

پرس‌وجوی شماره ۴ (فروش فروشندگان در سی روز اخیر) و پرس‌وجوی شماره ۶ (تعداد سفارش‌های هفت روز اخیر)، هر دو مستقیماً روی `order_date` با عبارت `INTERVAL - NOW()` فیلتر می‌کنند، اما برخلاف مسئله قبل، این بار `customer_id` در شرط `WHERE` حضور ندارد؛ این دو پرس‌وجو روی کل جدول Order اجرا می‌شوند، نه روی سفارش‌های یک مشتری. ایندکس (`customer_id, order_date DESC`) که برای مسئله اول ساخته شد، در این حالت کاربردی ندارد. دلیل آن این است که این ایندکس‌ها تنها زمانی به طور مؤثر استفاده می‌شوند که جست‌وجو از ستون اول (اینجا `customer_id`) شروع شود.

وقتی این ستون در پرس‌وجو غایب است، پایگاه داده نمی‌تواند از ایندکس برای محدود کردن جست‌وجو بر اساس order\_date استفاده کند.

## راه‌حل

یک ایندکس مستقل و ساده روی خود ستون order\_date تعریف شده است:

```
CREATE INDEX IDX_Order_Date  
ON "Order" (order_date);
```

این ایندکس امکان می‌دهد پایگاه داده به‌جای خواندن تمام ردیف‌های جدول Order، مستقیماً بازه زمانی موردنظر (هفت یا سی روز اخیر) را از طریق یک Range Scan روی ایندکس بیابد. با توجه به این‌که این دو پرس‌وجو ماهیت گزارش‌گیری دوره‌ای دارند و احتمالاً به‌طور مکرر (مثلاً هر روز یا هر بار بازدید از داشبورد فروش) اجرا می‌شوند، سرمایه‌گذاری روی این ایندکس در بلندمدت توجیه‌پذیر است.

## مسئله ۳: یافتن پرداخت‌های ناموفق یا معلق در میان انبوه پرداخت‌های موفق

پرس‌وجوی شماره ۵ به‌دنبال سفارش‌هایی است که وضعیت پرداخت آن‌ها Pending یا Failed باشد. ستون payment\_status در جدول Payment تنها چهار مقدار ممکن دارد (Pending, Successful, Failed, Refunded)، که در نگاه اول ممکن است این تصور را ایجاد کند که تنوع داده این ستون پایین است و طبق قاعده کلی، ایندکس‌گذاری روی آن سودمند نخواهد بود؛ دقیقاً همان استدلالی که صورت پروژه درباره ستون‌هایی مثل جنسیت مطرح کرده است. اما این استدلال کامل نیست، چون آنچه اهمیت دارد تعداد مقادیر ممکن یک ستون نیست، بلکه توزیع واقعی داده میان آنهاست. در یک فروشگاه آنلاین واقعی، اکثر پرداخت‌ها در نهایت با وضعیت Successful ثبت می‌شوند و تنها بخش کوچکی به Failed یا Pending می‌رسند. پرس‌وجوی شماره ۵ دقیقاً همین بخش کوچک را هدف می‌گیرد. در چنین حالتی، با وجود این‌که ستون تنها چهار مقدار دارد، فیلتر کردن روی دو مقدار خاص (Pending و Failed) می‌تواند نسبت کوچکی از کل جدول را برگرداند و همین موضوع باعث می‌شود ایندکس مفید باشد، برخلاف حالتی که مقادیر یک ستون تقریباً به‌طور مساوی در جدول پخش شده باشند.

## راه‌حل

یک ایندکس ساده روی ستون payment\_status تعریف شده است:

```
CREATE INDEX IDX_Payment_Status  
ON Payment (payment_status);
```

این پرس‌وجو معمولاً توسط تیم پشتیبانی سفارش برای پیگیری شکایات مشتریان و بررسی پرداخت‌های ناموفق اجرا می‌شود و انتظار می‌رود با تناوب بالایی فراخوانی شود، بنابراین هزینه نگهداری ایندکس در برابر سرعت پاسخ‌دهی به این پرس‌وجو قابل توجیه

است. لازم به ذکر است این استدلال به‌طور عمومی برای همه ستون‌های وضعیت در پروژه تعمیم داده نشده؛ برای ستونی مانند `order_status` که مقادیرش (Pending, Processing, Shipped, Delivered, Cancelled) توزیع متعادل‌تری دارند و به‌تنهایی در هیچ‌کدام از پرس‌وجوهای تست به‌عنوان تنها شرط فیلتر ظاهر نمی‌شوند، ایندکس مجزایی تعریف نشده است.

#### مسئله ۴: شرط مقایسه بین دو ستون از یک ردیف

پرس‌وجوی شماره ۳ محصولاتی را نمایش می‌دهد که `stock_quantity` آن‌ها کمتر از `alert_threshold` باشد. در این حالت، یک ایندکس B-Tree معمولی، چه روی `stock_quantity` و چه روی `alert_threshold`، توسط بهینه‌ساز PostgreSQL قابل استفاده نیست. علت این است که ایندکس B-Tree بر پایه مقایسه هر مقدار با یک مرز مشخص کار می‌کند، در حالی که در این پرس‌وجو مرز مقایسه ثابت نیست و از ستون دیگر همان ردیف گرفته می‌شود. به همین دلیل، ساخت یک ایندکس معمولی برای این مسئله به‌خودی‌خود کارساز نیست.

#### راه‌حل

با توجه به این‌که این پرس‌وجو یکی از کوئری‌های عملیاتی پرکاربرد سیستم انبار است (و در گام پنجم پروژه هم مبنای `view` با نام `inventory_alerts_vw` قرار گرفته)، و با توجه به این واقعیت که در عمل تنها بخش کوچکی از محصولات در هر لحظه زیر آستانه هشدار قرار می‌گیرند، راه‌حل انتخاب‌شده استفاده از یک `Partial Index` است:

```
CREATE INDEX IDX_Inventory_LowStock
ON Inventory_Stock (product_id)
WHERE stock_quantity < alert_threshold;
```

این ایندکس تنها شامل ردیف‌هایی می‌شود که در لحظه ساخت یا به‌روزرسانی، موجودی‌شان کمتر از آستانه هشدار است؛ در داده‌های تستی فاز سوم، این تعداد سه محصول از ده محصول موجود است. نتیجه آن، حجم بسیار کوچک ایندکس نسبت به کل جدول است، و هر بار که پرس‌وجوی شماره ۳ اجرا شود، پایگاه داده می‌تواند به‌جای بررسی تک‌تک محصولات، مستقیماً از این ایندکس کوچک بخواند.

نکته‌ای که باید در نظر داشت این است که چون `stock_quantity` در پی هر عملیات فروش، بازگشت یا رسید کالا تغییر می‌کند، محتوای این `Partial Index` هم باید به‌طور مداوم به‌روزرسانی شود؛ یعنی هزینه نگهداری این ایندکس صفر نیست. اما چون تعداد ردیف‌هایی که در هر لحظه شرط `WHERE` ایندکس را برآورده می‌کنند معمولاً کم است، این هزینه در مقایسه با کوچک بودن حجم ایندکس و سرعت بازیابی، منطقی و قابل قبول است.

#### مسئله ۵: کندی JOIN زنجیره‌ای در محاسبه فروش هر فروشنده

پرسوجوی شماره ۴ برای محاسبه مجموع فروش هر فروشنده، یک زنجیره چهارتایی از JOIN را طی می‌کند: از Seller به Product، از Product به Order\_Item، و از Order\_Item به Order. نقطه حساس این زنجیره، اتصال Product به Order\_Item از طریق ستون product\_id است.

در جدول Order\_Item، کلید اصلی ترکیبی (order\_id, item\_number) است. ستون product\_id در این جدول یک کلید خارجی است، اما چون در کلید اصلی حضور ندارد، ایندکس ضمنی کلید اصلی که با order\_id شروع می‌شود برای جستوجوی بر اساس product\_id مفید نیست. در نتیجه برای هر محصول یک فروشنده، پیدا کردن تمام آیتم‌های سفارشی که آن محصول در آن‌ها فروخته شده، بدون ایندکس مناسب به یک جستوجوی خطی روی جدول Order\_Item منجر می‌شود؛ عملیاتی که با افزایش تعداد سفارش‌ها و آیتم‌های سفارش در طول زمان، اثر منفی روی سرعت این گزارش می‌گذارد.

## راه‌حل

یک ایندکس ترکیبی روی (product\_id, order\_id) تعریف شده است:

```
CREATE INDEX IDX_OrderItem_Product_Order
ON Order_Item (product_id, order_id);
```

ستون product\_id به‌عنوان بخش اول این ایندکس، امکان می‌دهد پایگاه داده به طور مستقیم تمام آیتم‌های سفارش مربوط به یک محصول را بیابد. افزودن order\_id به‌عنوان بخش دوم ایندکس هم به این دلیل انتخاب شده که مرحله بعدی زنجیره JOIN (اتصال به جدول Order) دقیقاً بر پایه همین order\_id انجام می‌شود؛ به این ترتیب بخشی از این JOIN هم می‌تواند مستقیماً از داده‌های ذخیره‌شده در خود ایندکس پاسخ داده شود، بدون نیاز به مراجعه اضافی به جدول اصلی.